

认识 AI 从模型到工具

从 AI 的技术分层出发，理解程序员的沟通角色，再通过真实项目对话认识技术语言的价值。

14 个学习章节

概念关系

真实项目案例

技术语言与 AI 协作

01 两条主线：技术认知与效能工具

本教程围绕“AI 的技术认知和效能工具”展开，分别讨论如何理解 AI，以及如何在工作中使用 AI。技术基础仍然重要：例如判断 AI 是否错误修改了过滤器，需要理解原有代码与业务行为。

技术认知部分回应学习与职业发展中的疑问，包括未来技术走向、继续学习的价值，以及前端等方向的发展。这些疑问往往伴随焦虑和迷茫。

原课认为，部分焦虑来自对技术缺乏理解。就像接触一个陌生框架时，未知会增加不安；建立清晰的知识结构后，更容易判断它能解决什么问题。本教程首先梳理 AI 的技术格局。

效能工具部分主要围绕 AI 编程，讨论工作实践中的使用方法。原课观察到，AI 已经参与大量开发工作；军工或涉密机构等内网环境可能不便使用这类工具，这里不作展开。

即使使用同一款 AI 工具处理相同类型的项目，不同人的效果也可能差异很大。需要关注工具选择、沟通方式、人的角色，以及不同问题应采用的协作方法。

02 面对新名词，先看技术结构

AI 技术认知不仅用于理解变化，也用于建立知识体系。若主要通过“炸裂”“震惊”“颠覆”等宣传性表述获取信息，容易高估单个新概念的影响。

从 2023 年至原课录制时，新名词持续出现。相比逐个记忆名称，更重要的是形成技术结构，理解概念之间的关系。

本教程将 AI 技术结构分为裸模型、模型服务、AI 应用三层，并按这一顺序展开。最底层是裸模型，即这里讨论的各类大模型。



讲者列举了大量 AI 技术名词，说明仅记忆名词难以建立清晰的知识体系。

03 第一层：裸模型

GPT-5.2、Kimi、智谱模型和 Gemini 是原课列举的模型示例。注意力机制、KV-Cache、上下文窗口等概念也放在裸模型层讨论。



裸模型一行列出模型名称、注意力机制、KV-Cache 和上下文窗口。

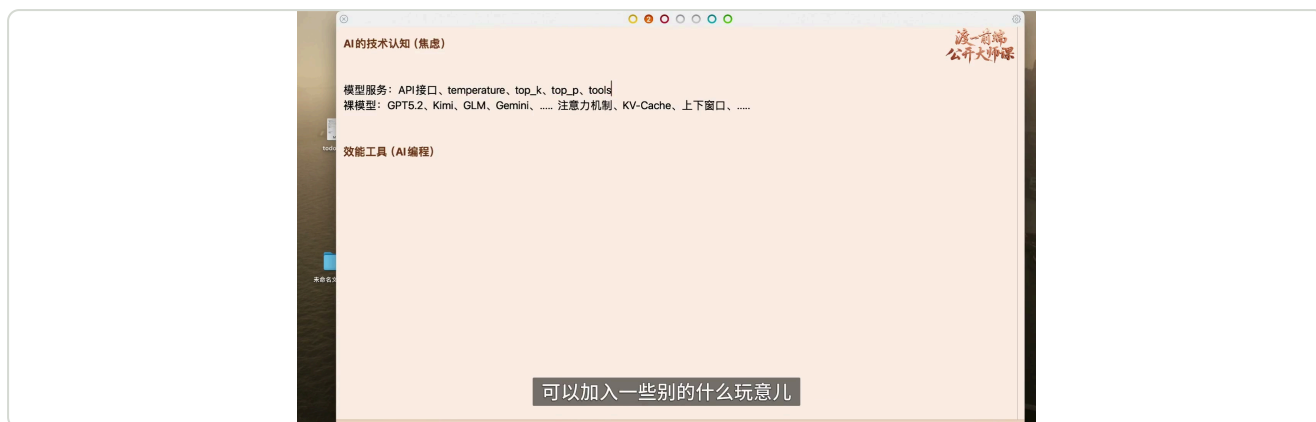
画面中的细节

- 画面中出现的模型名称：GPT5.2、Kimi、GLM、Gemini。
- 模型相关术语：注意力机制、KV-Cache、上下文窗口。

04 第二层：模型服务

模型服务是对裸模型能力的封装，通常由服务商以 API 形式提供。原课以 OpenAI 提供 GPT 模型服务为例，说明模型能力如何向外部程序开放。

程序按照远程接口的协议发送问题、获取回复。本教程将 temperature、top_k、top_p、tools 等放在模型服务层讨论，以区分服务参数与模型内部机制。



模型服务层列出 API 接口及 temperature、top_k、top_p、tools 等。

画面中的细节

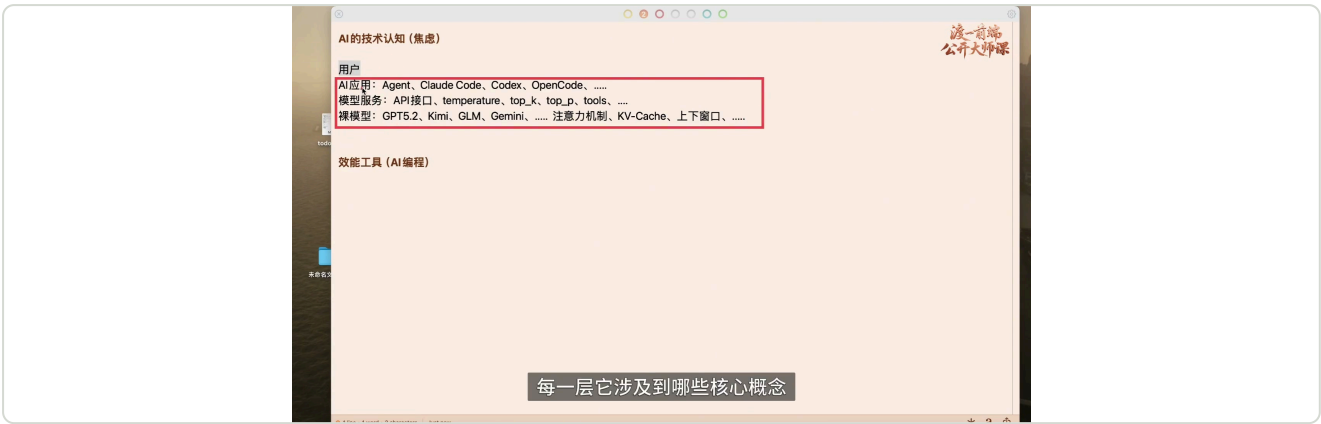
- API 参数按原板书保留为 temperature、top_k、top_p、tools。

05 第三层：AI 应用与学习路径

AI 应用在模型服务之上继续封装，调用接口提供具体功能。Agent、Claude Code、Codex、OpenCode 及其他 AI 编程应用，都在这一层讨论；用户位于应用层之上。

直接用于 AI 编程的效能工具属于应用层。若要开发 AI 应用，需要理解模型服务接口，以及应用层常见的概念和开发方法。

模型微调涉及对模型本身的调整，因此属于裸模型层。建立三层结构后，可以判断新概念所在的层次及其作用，再学习相应内容。



完整板书：用户、AI 应用、模型服务、裸模型。

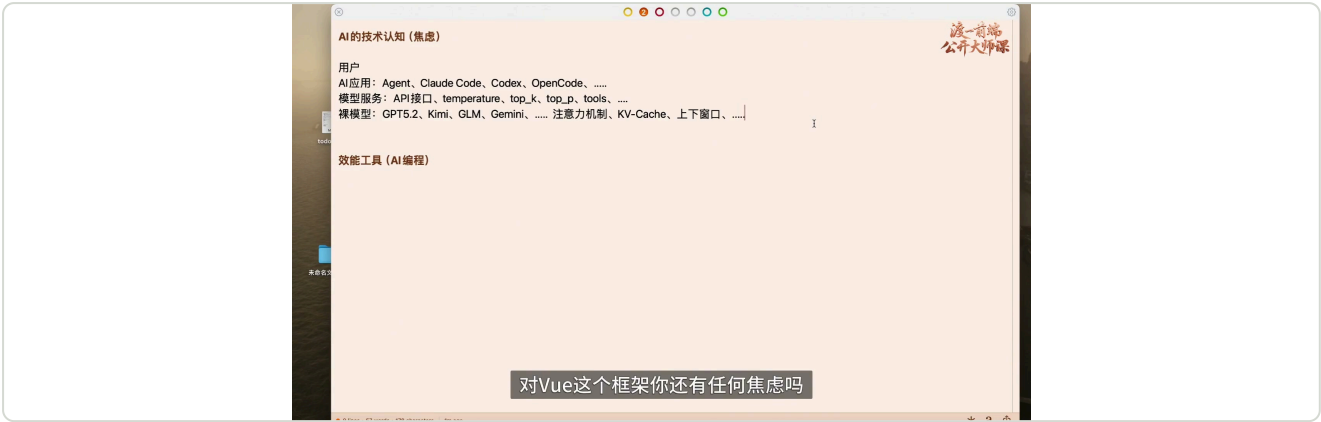
画面中的细节

- 应用层示例：Agent、Claude Code、Codex、OpenCode。
- 用户位于应用层上方；使用工具、开发应用、微调模型对应不同关注层次。

06 认知差异，决定使用差异

学习 Vue、React 等框架时，对原理和使用方式的理解会影响实际效果。AI 工具也一样：理解越充分，越容易作出正确判断；概念模糊和知识盲区则可能影响使用效果。

例如，能够理解甚至实现 Vue 的核心源码，有助于处理框架相关问题和实际场景。这个类比说明，深入理解可以减少对未知的焦虑。



通过框架学习的类比，说明理解深度与使用效果的关系。

07 程序员是需求与运行时之间的桥梁

AI 改变了程序员的工作方式，但原课认为，连接现实需求与技术实现的核心角色仍然存在。

程序员承担沟通与转换的职责，在现实世界和计算机运行时之间建立联系。运行时是执行代码的环境，例如浏览器或服务器。

需求可能来自产品经理、管理者或开发者自身。程序员理解并加工这些需求，将其转换为运行环境能够执行的代码，再接收执行结果。

程序员需要判断结果是否满足需求，必要时继续沟通和调整。加入 AI 后，这一职责仍然存在，因为人与 AI 的协作虽然使用自然语言，却仍需要技术背景与技术表达。



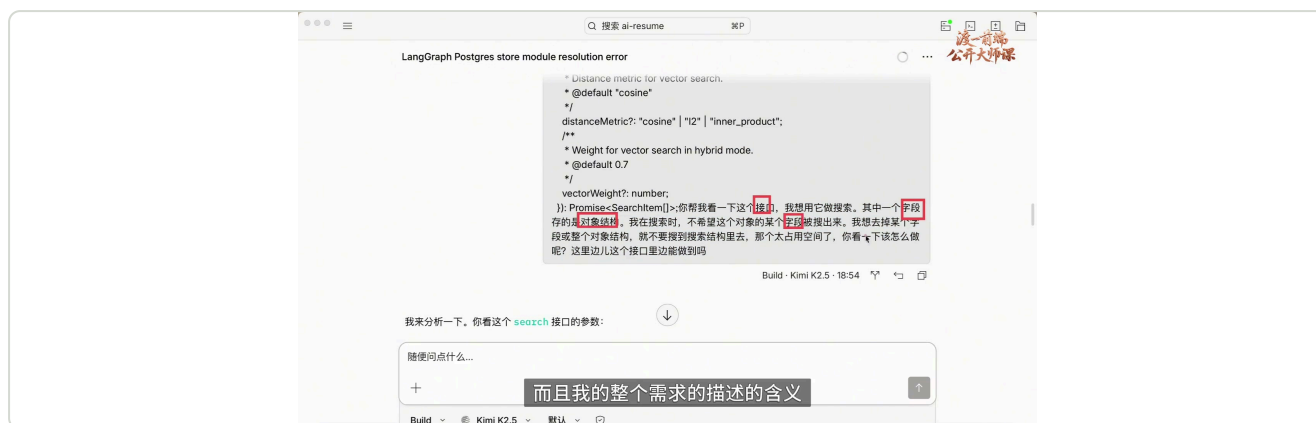
现实世界、沟通者和计算机运行时构成需求实现与反馈链路。

08 案例一：用技术语言分析搜索接口

案例一将一个参数较多的函数定义交给 AI 分析，确认现有搜索接口能否满足特定需求。表面上是自然语言提问，实际依赖对接口能力的理解。

需求涉及字段和对象结构：某个字段存放了对象，搜索时希望排除其中的子字段或整个对象，避免返回结果体积过大，需要确认接口是否支持以及如何实现。

函数定义来自项目代码。只有理解其含义，才能准确描述问题并继续追问。下一个案例使用语音输入反馈问题，文字识别虽不够整齐，但技术要求仍然明确。



搜索接口提问：如何避免把某个字段或对象结构带入搜索结果。

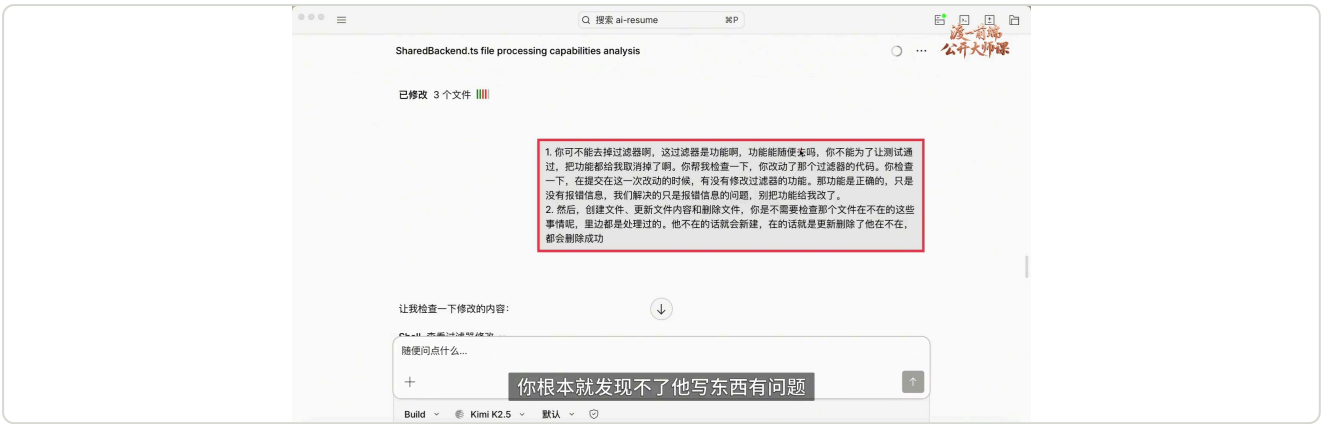
画面中的细节

- 示例来自 ai-resume 项目，界面标题为 “LangGraph Postgres store module resolution error”。
- 函数定义中可见 distanceMetric 的候选值 cosine、l2、inner_product，以及 vectorWeight。
- 提问要求判断现有 search 接口能否控制返回字段，并没有在本课展示完整解决方案。

09 案例二：测试通过，功能仍可能被改坏

案例二中，AI 报告修复完成且测试通过，但修改过程改变了过滤器功能。测试通过不能代替业务行为验证；发现这种偏差，需要理解原有功能和改动内容。

因此，使用 AI 并不意味着可以省去技术学习。后续案例还涉及把复杂实现整理为文档，而模型可能不了解训练之后出现的库和技术，需要结合项目代码分析。



过滤器属于功能，不能为了测试通过而直接移除或改变其行为。

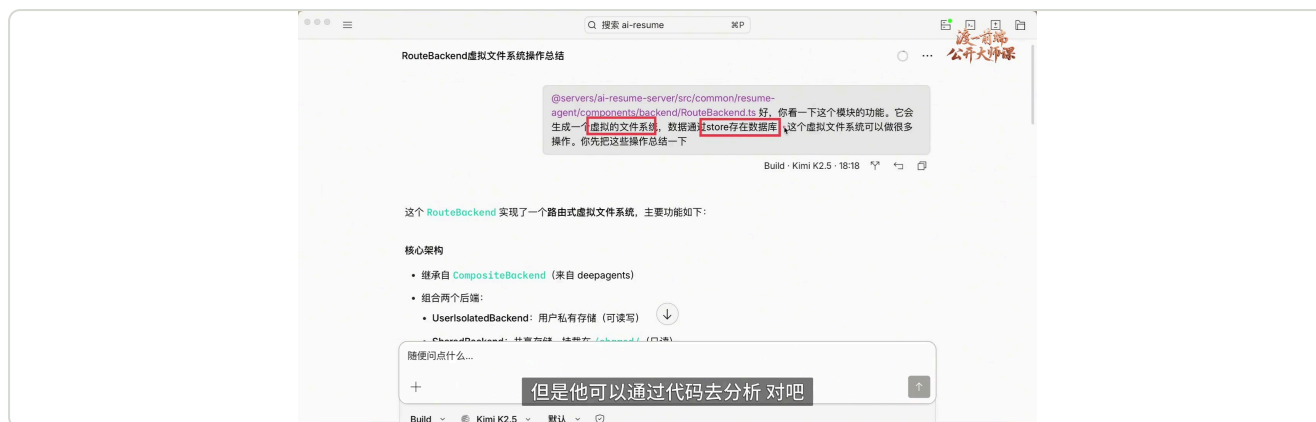
画面中的细节

- 第一点：检查改过的过滤器代码，只解决报错信息问题，不要改掉原有功能。
- 第二点：创建、更新、删除文件的存在性处理，项目内部已有约定；不能重复加入不符合约定的判断。
- 这些要求属于该项目的语义，不能直接推广为所有文件系统都适用的规则。

10 案例三：理解代码，再沉淀文档

例如，假设模型于 2025 年训练，它可能不了解之后出现的技术。将复杂实现整理为文档时，需要提供关键背景，让 AI 阅读代码、分析逻辑并形成说明。

案例中的关键背景是一个虚拟文件系统：数据通过 store 存入数据库。无需逐项解释能够从代码推导的信息，但需要指出核心设计。之后还应检查 AI 返回的对象结构，针对疑点继续追问。



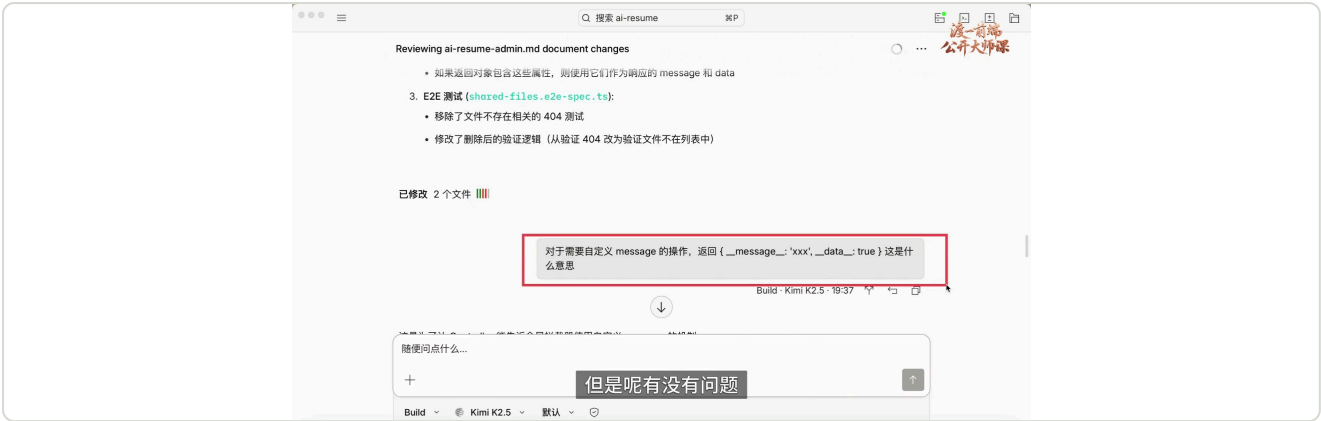
RouteBackend.ts 的虚拟文件系统说明与文档整理请求。

画面中的细节

- 请求明确指向 `servers/ai-resume-server/src/common/resume-agent/components/backend/RouteBackend.ts`。
- 讲者提示：生成虚拟文件系统，数据通过 store 存入数据库，请先总结这些操作。
- AI 回复画面中提到 `CompositeBackend`、`UserIsolatedBackend` 和 `SharedBackend`。本课没有进一步讲解完整实现。

11 案例四：发现疑问，追问返回结构

案例四中的返回结构最终被确认没有问题，但这一结论来自追问和验证。技术理解帮助开发者识别疑点、提出问题，并判断解释是否成立。



追问返回对象中 __message__、__data__ 的含义。

画面中的细节

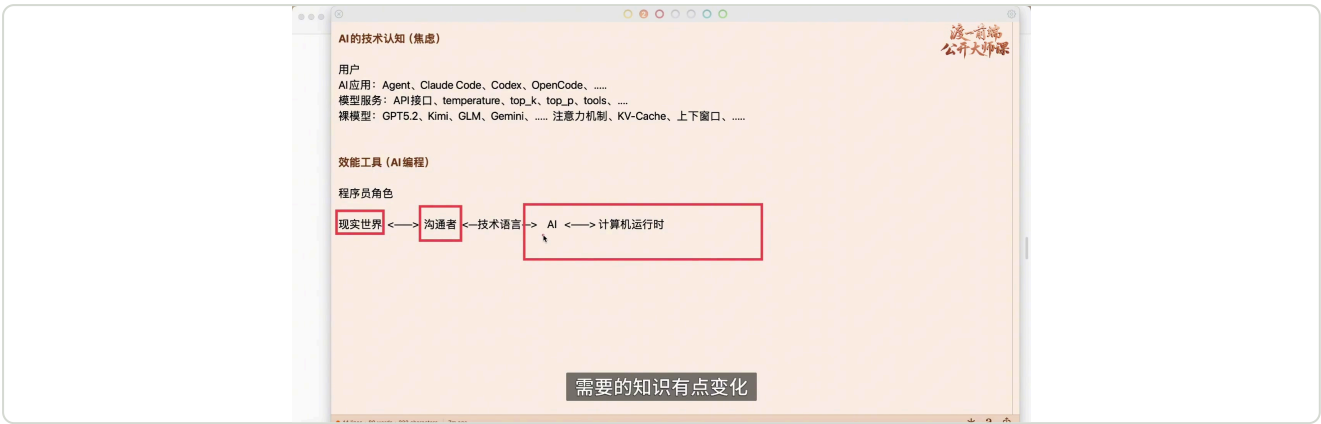
- 画面中的返回示例：{ '__message__': 'xxx', '__data__': true }。
- 上方说明：返回对象含这些属性时，用作响应的 message 和 data。
- 画面同时提到 E2E 测试 shared-files.e2e-spec.ts：移除文件不存在的 404 测试，删除后的验证改为检查文件不在列表中。这是项目案例细节。

12 商业应用需要持续的技术沟通

俄罗斯方块或贪吃蛇等简单示例，与真实商业应用的复杂度不同。商业项目可能包含几万、几十万甚至上百万行代码，需要持续澄清需求、评估结果并调整实现，这些过程离不开技术语言。

程序员仍然是现实需求与技术实现之间的桥梁。所需知识会变化，但理解越深、知识越广，越有利于与 AI 有效协作。

充分的技术理解有助于给 AI 提供准确的指令、方向和步骤。原课以“招聘要求并未降低”的个人观察，进一步说明判断和引导工具仍然需要专业能力。



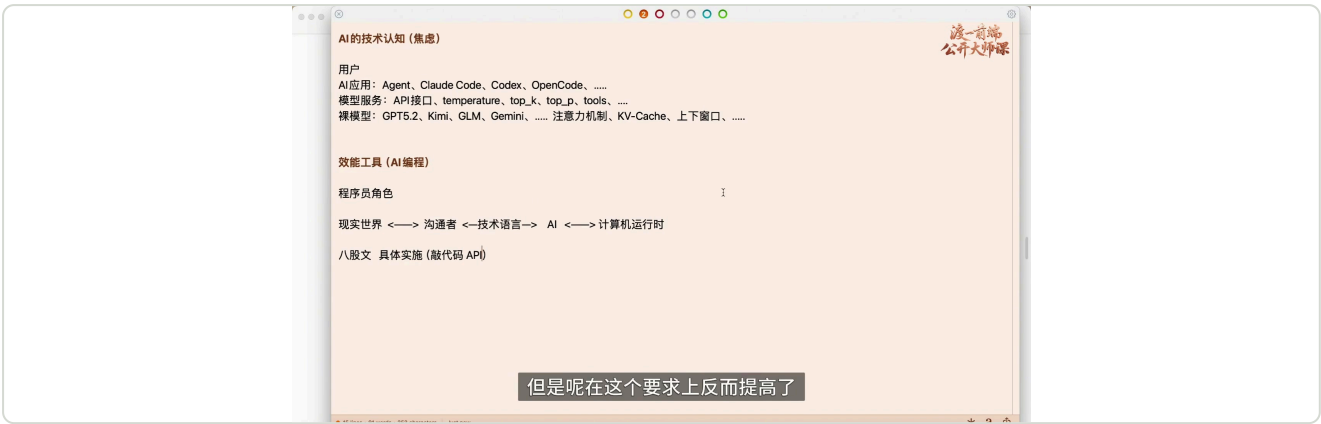
AI 加入协作链路后，人与 AI 之间仍通过技术语言沟通。

13 能力要求的重心正在变化

原课通过一个假设作比较：若 AI 已完全替代技术判断，企业或许只需要会聊天和使用工具的低门槛岗位。但讲者观察到的要求反而更高，并据此强调知识储备对协作的重要性。

相对降低的可能是对手工逐行编写代码、记忆某些 API、函数和对象使用细节的依赖。原课将这些与思维和体系性理解作区分，并不认为所有技术要求都会消失。

相对更受重视的是思维、认知以及知识体系的广度和深度。这里关于招聘与能力要求的讨论，保留为讲者的观察和推论。



板书区分具体实施、API 使用等细节与更广泛的技术认知。

画面中的细节

- “招聘要求没有降低”及其原因，是课程讲者的观察与推论，没有在原视频中提供统计数据。

14 让认知与工具使用相互促进

AI 编程实践需要选择适合个人与项目的工具，并采用有效的沟通方式。技术认知与工具使用相互促进：理解更准确，通常更容易用好工具。

了解 AI 擅长的任务、能力限制及容易出错的环节，有助于在协作中提前识别问题。本教程将沿技术认知与实际使用两条主线展开。



两条主线最终回到同一目标：理解 AI，并更好地使用它。